Hàm

C++ Cơ Bản · 28Tech

https://blog.28tech.com.vn/cpp-ham-function

Hàm giải quyết vấn đề gì?

- Khối lệnh thực hiện **1 chức năng cụ thể**, dùng lại được nhiều lần
- Tránh code lặp lại, chương trình dễ quản lý hơn khi lớn
- Gọi hàm nhiều lần thay vì viết lại toàn bộ logic

Cú pháp khai báo hàm

- `data_type`: kiểu trả về (`int`, `void`, `long long`,...)
- `function_name`: đặt tên theo quy tắc như tên biến
- `parameter`: đầu vào của hàm, số lượng và kiểu tùy ý

```
data_type function_name(type1 param1, type2 param2){
    // code
}

int tong(int a, int b, int c){
    return a + b + c;
}
```

Lời gọi hàm — luồng chạy

- Gọi hàm trong `main` (hoặc hàm khác) để nó thực thi
- Chương trình **nhảy vào** thân hàm, chạy hết lệnh rồi **quay lại**
- Mỗi lần gọi, các câu lệnh trong hàm chạy lại từ đầu

```
void greet(){
    cout << "Hello 28tech C++!\n";
}

int main(){
    cout << "Before\n";
    greet();          // nhảy vào thân hàm
    cout << "After\n";
}
// Before / Hello 28tech C++! / After
```

Tham số và đối số

- **Tham số** (parameter): khai báo trong định nghĩa hàm
- **Đối số** (argument): giá trị truyền vào khi gọi hàm
- Đối số gán cho tham số theo **đúng thứ tự** khai báo
- Kiểu đối số nên trùng hoặc nhỏ hơn kiểu tham số, tránh mất dữ liệu

Tham số truyền theo giá trị

- Thay đổi tham số **bên trong hàm không ảnh hưởng** đối số bên ngoài

```
void thaydoi(int n){
    n += 28;
    cout << n << endl;      // 128
}

int main(){
    int a = 100;
    thaydoi(a);
    cout << a << endl;      // vẫn là 100
}
```

Câu lệnh return

- Trả **giá trị** cho nơi gọi hàm, hàm kết thúc **ngay khi gặp return**
- Kiểu giá trị `return` nên khớp kiểu trả về của hàm
- Hàm `void` dùng `return;` (không kèm giá trị) để thoát sớm

```
int tong(int a, int b){
    return 28;      // kết thúc, trả 28 ngay
    return a + b;   // không bao giờ chạy tới
}

void greet(int n){
    cout << n << endl;
    return;         // thoát sớm khỏi hàm void
    cout << "khong chay toi day\n";
}
```

Ví dụ: hàm có return

```
int tong(int a, int b, int c){
    return a + b + c;
}

long long factorial(int n){
    long long gt = 1;
    for(int i = 1; i <= n; i++) gt *= i;
    return gt;
}
```

Xây dựng hàm tổng quát

- Hàm chỉ nên làm **1 nhiệm vụ rõ ràng**, tránh gộp nhập/tính/xuất
- Nhập, xuất nên để ở `main`; hàm chỉ xử lý và `return` kết quả
- Hàm tổng quát tái sử dụng được cho nhiều bài toán khác nhau

```
// chưa tổng quát: gộp cả nhập và xuất vào hàm
void xuly(){
    int n; cin >> n;
    int tong = 0;
    for(int i=1;i<=n;i++) if(n%i==0) tong+=i;
    cout << tong << endl;
}

// tổng quát: chỉ tính và trả kết quả
int tonguoc(int n){
    int tong = 0;
    for(int i=1;i<=n;i++) if(n%i==0) tong+=i;
    return tong;
}
```

Ghi nhớ

- Hàm giúp tái sử dụng code, chương trình dễ quản lý hơn
- Tham số truyền theo giá trị: sửa trong hàm không đổi đối số ngoài
- `return` giá trị kết thúc hàm ngay lập tức
- Hàm nên tổng quát, tách biệt nhập/xuất khỏi phần xử lý